INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Departamento de Engenharia Informática

Inteligência Artificial

Primeiro Trabalho Prático

Jogo ISOLA em Prolog

Minimax e Alfa-Beta

Diogo Oliveira, Nº 51662

Paulo Nascimento, Nº 51665

Grupo 4

Semestre de Verão 2025/2026

Professor: Dr. Nuno Miguel da Costa de Sousa Leite

Data de Entrega: 20 de abril de 2026

Índice

1 Introdução 6
2 Descrição do jogo ISOLA 7
2.1 Regras do Jogo 7
2.2 Representação do Estado 7
3 Estrutura do Código 8
4 Principais Predicados 9
4.1 main.pl 9
4.2 utils.pl 9
4.3 replace.pl 10
4.4 print.pl 10
5 Algoritmos de Pesquisa Adversarial 11
5.1 Minimax 11
5.2 Alfa-Beta 11
5.2.1 Função de Avaliação Estática 11
5.2.2 Otimizações da Implementação 12
5.3 Comparação Minimax vs Alfa-Beta 13
6 Avaliação Experimental 14
6.1 Tabuleiro 3x3 Minimax sem limite 14
6.2 Tabuleiro 6x6 Alfa-Beta com profundidade 3 14
6.3 Impacto da profundidade no Alfa-Beta 14
6.4 Testes 16
7 Decisões de Design e Justificações 25
7.1 Separação modular do código 25
7.2 Mobilidade como heurística de avaliação 25
7.3 Profundidade de pesquisa a 3 25
7.4 Validação tolerante das jogadas do utilizador 25
7.5 Utilização de findall/3 para geração de sucessores 25
8 Conclusão 26

Índice de tabelas

Tabela 1 Descrição de cada ficheiro 8
Tabela 2 Comparação Minimax vs Alfa-Beta 13
Tabela 3 Tabela de profundidade/tempo tabuleiro 4x4 (melhor caso) 14
Tabela 4 Tabela de profundidade/tempo tabuleiro 4x4 (pior caso) 15

Índice de figuras

Figura 1 memória usada pelo prolog 15
Figura 2 Vitória do minimax 16
Figura 3 vitória do alfa-beta 17
Figura 4 Jogada invalida do humano 17
Figura 5 Jogo perfeito por parte da AI parte 1 18
Figura 6 Jogo perfeito por parte da AI part 2 19
Figura 7 Jogo perfeito por parte da AI part 3 20
Figura 8 Jogo perfeito por parte da AI part 4 21
Figura 9 alfa-beta com 6 de profundidade 21
Figura 10 alfa-beta com 5 de profundidade 22
Figura 11 alfa-beta com 4 de profundidade 23
Figura 12 alfa-beta com 4 de profundidade 24

1 Introdução

O presente relatório descreve o desenvolvimento do Primeiro Trabalho Prático da Unidade Curricular de Inteligência Artificial, do Semestre de Verão 2025/2026.

O objetivo do trabalho consistiu em implementar o jogo ISOLA em linguagem Prolog, integrando algoritmos clássicos de pesquisa adversarial, o algoritmo Minimax e a sua otimização, o algoritmo Alfa-Beta. O jogo suporta dois modos: Humano vs. Humano e Humano vs. Computador (IA).

O jogo ISOLA é um jogo de estratégia para dois jogadores, criado por Bernd Kienitz em 1972, onde o objetivo é isolar o peão adversário impedindo-o de se mover.

2 Descrição do jogo ISOLA

2.1 Regras do Jogo

O ISOLA é disputado num tabuleiro quadrado onde cada jogador possui um peão. O objetivo é isolar o peão adversário de modo que este não tenha movimentos válidos em nenhuma direção (horizontal, vertical ou diagonal).

Turno de Jogo:

- Mover: O jogador desloca o seu peão uma casa para qualquer posição adjacente livre (as 8 direções possíveis).
- Remover: O jogador escolhe e remove qualquer casa vazia do tabuleiro, marcando-a com '#'.

Restrições:

- Não é permitido entrar em casas marcadas com '#' (removidas).
- Não é possível saltar por cima do adversário.
- Se um jogador ficar sem movimentos válidos, perde a partida.

2.2 Representação do Estado

O tabuleiro é representado como uma lista de listas em Prolog. Cada célula pode conter um de três valores:

'x' — Posição do Jogador 1 (humano)

'o' — Posição do Jogador 2 (computador / IA)

'.' — Casa vazia disponível

'#' — Casa removida (bloqueada)

Exemplo de tabuleiro inicial 4x4:

[[x,.,.,.],

[.,.,.,.],

[.,.,.,.],

[.,.,.,o]].

3 Estrutura do Código

O projeto está organizado em 6 ficheiros Prolog, cada um responsável por uma funcionalidade específica:

Tabela 1 Descrição de cada ficheiro

Ficheiro	Descrição
`main.pl`	Ponto de entrada. Gere o ciclo principal do jogo, a alternância de jogadores e a interação com o utilizador.
`minimax.pl`	Implementação do algoritmo Minimax clássico (sem profundidade limitada).
`Alfa_beta.pl`	Implementação do algoritmo Alfa-Beta com profundidade limitada e função de avaliação estática melhorada.
`utils.pl`	Predicados utilitários usados em outros ficheiros.
`print.pl`	Predicados de impressão do tabuleiro no terminal.
`replace.pl`	Predicados de substituição de elementos em listas e tabelas.

4 Principais Predicados

Os comentários de cada predicado tiveram o auxílio de ferramentas de AI, baseandos em rascunhos preexistentes. O objetivo foi padronizar a documentação seguindo o estilo KDoc, suprindo a pouca experiência na sintaxe de comentários nativa do Prolog.

4.1 main.pl

`play(RoundNumber, Board) :-`

O predicado principal play/2 recebe o número do turno e o tabuleiro atual. A cada turno determina qual o jogador ativo (par = 'x', ímpar = 'o'), verifica se está bloqueado e, se não estiver, executa a jogada correspondente.

Decisões de design tomadas:

- O jogador 'x' é sempre o humano e introduz as suas jogadas manualmente.
- O jogador 'o' é controlado pelo computador via Alfa-Beta ou minimax.
- Após cada movimento do humano, é invocado setupSharp/2 para pedir ao jogador que remova uma casa.
- Validação de jogadas inválidas com reemissão do turno.

4.2 utils.pl

`indexOf(Board, Element, Row, Col) :-`

Este predicado tira partido da reversibilidade do Prolog, assumindo três comportamentos distintos consoante a instanciação dos seus argumentos:

- Se as coordenadas (Row e Col) forem fornecidas, o predicado unifica a variável Element com o conteúdo presente nessa posição do tabuleiro.
- Caso seja fornecido apenas o Element, o predicado percorre a matriz do tabuleiro e devolve a linha e coluna onde este se encontra.
- Se todos os argumentos forem fornecidos, o predicado atua como uma verificação, confirmando se o elemento especificado existe exatamente nas coordenadas indicadas.

`isValidPlay(X, Y, Player, Board, OldX, OldY) :-`

- O destino está dentro dos limites da vizinhança (adjacência de 1 casa).
- A casa de destino é um espaço neutro ('.').
- O movimento não é nulo (não pode saltar sobre si próprio).

`isBlocked(Player, Board) :-`

- Utiliza o conceito de negação por falha (\+).
- O sistema tenta encontrar pelo menos uma coordenada vizinha que satisfaça isValidPlay. Se falhar em encontrar qualquer saída, o predicado confirma que o jogador está bloqueado, determinando a sua derrota.

```
setupSharp(Board, NewBoard) :-
```

- Em vez de alterar o tabuleiro original, o putSharp gera uma nova versão do tabuleiro (NewBoard) com o obstáculo # inserido.
- O setupSharp lida com o erro humano. Se o jogador escolher uma casa inválida, o predicado não termina; ele volta a chamar-se a si próprio até que uma jogada válida seja submetida.

4.3 replace.pl

O predicado replaceTable/5 substitui o valor de uma célula específica do tabuleiro. Internamente usa replace/4 que percorre recursivamente uma lista até ao índice desejado.

4.4 print.pl

Os predicados printList/1 e printTable/1 percorrem recursivamente o tabuleiro e imprimem cada valor, com os elementos separados por espaços.

5 Algoritmos de Pesquisa Adversarial

5.1 Minimax

O algoritmo Minimax foi implementado no ficheiro minimax.pl segundo o modelo clássico. Explora a árvore de jogo até ao fim (sem limite de profundidade), o que o torna perfeito, mas demasiado lento para tabuleiros maiores.

Predicados principais:

- minimax/3 - avalia a melhor posição sucessora de uma dada posição.
- best/3 - percorre a lista de posições candidatas e retorna a melhor.
- betterof/6 - compara dois estados e devolve o melhor segundo o papel do jogador (MAX ou MIN).
- staticval/2 - avalia estaticamente um estado terminal: +1000 se 'o' ganhou, -1000 se 'x' ganhou.
- moves/2 - gera todos os estados sucessores válidos de uma posição.
- auxMoves/2 - para cada direção possível, executa o movimento e a remoção de uma casa, gerando o estado resultante.

5.2 Alfa-Beta

O algoritmo Alfa-Beta, no ficheiro Alpha_beta.pl, é uma implementação eficiente do Minimax que diminui a quantidade de ramos possíveis. Inclui um limite de profundidade (configurado para 3 no código atual) e uma função de avaliação estática melhorada.

Predicados principais:

- Alfabeta/7 - predicado central que recebe a posição, os limites Alfa/beta, a profundidade atual e máxima.
- boundedbest/7 - percorre a lista de candidatos dentro dos limites alfa-beta.
- goodenough/9 - decide se deve continuar a explorar ou pode podar o ramo atual.
- newbounds/6 - atualiza os limites Alfa e beta conforme o papel do jogador.
- staticval/2 - função de avaliação estática melhorada, que usa a diferença de mobilidade entre os dois jogadores em vez de apenas valores terminais.
- mobilidade/3 - conta o número de movimentos válidos disponíveis para um jogador numa dada posição, sendo o critério central da função de avaliação.

5.2.1 Função de Avaliação Estática

A função de avaliação estática é o componente responsável por atribuir um valor numérico a estados não terminais da árvore de jogo, permitindo ao algoritmo Alfa-Beta tomar decisões

quando a profundidade máxima de pesquisa é atingida. No contexto do jogo ISOLA, esta função foi concebida em torno do conceito de mobilidade o número de movimentos válidos disponíveis a cada jogador por se tratar de uma métrica diretamente correlacionada com a condição de vitória (bloquear o adversário).

Val = Mobilidade('o') - Mobilidade ('x')

Esta formulação é natural dada a convenção adotada: o computador (o) é o maximizador e o humano (x) é o minimizador. Valores positivos favorecem o computador, valores negativos favorecem o humano, e valores próximos de zero correspondem a estados equilibrados. O computador procura simultaneamente maximizar a sua própria mobilidade e minimizar a do adversário, o que traduz diretamente a estratégia ganhadora do jogo.

Para estados terminais, em que um dos jogadores fica sem movimentos legais, são atribuídos valores extremos que dominam qualquer avaliação heurística:

- Mobilidade(o) = 0 → Val = −1000 (o computador perdeu)
- Mobilidade(x) = 0 → Val = +1000 (o computador ganhou)

A magnitude destes valores (±1000) foi escolhida de forma a garantir que nenhuma diferença heurística de mobilidade intermédia possa sobrepor-se a um estado terminal na comparação entre ramos, assegurando que o algoritmo prefere sempre uma vitória garantida a qualquer posição intermédia, ainda que heuristicamente favorável.

Para melhorar o tempo de execução, recorreu-se a ferramentas de IA na otimização da função heurística, uma vez que o âmbito do projeto não exigia uma heurística de elevado desempenho.

5.2.2 Otimizações da Implementação

A implementação do algoritmo Alfa-Beta no ficheiro alpha_beta.pl inclui várias otimizações que vão além do pseudocódigo clássico, orientadas sobretudo para mitigar o elevado fator de ramificação do jogo ISOLA, resultante da combinação do movimento do peão com a remoção de uma casa em cada turno.

5.2.2.1 Poda agressiva na geração de sucessores

O predicado auxMoves/2 foi modificado face à versão do Minimax para não gerar todos os estados possíveis. Em vez de considerar qualquer casa vazia como candidata à remoção, apenas são geradas remoções em casas adjacentes ao peão adversário, isto é, casas que correspondem a movimentos legais do oponente no próximo turno. Esta decisão reduz drasticamente o número de sucessores gerados sem comprometer a qualidade estratégica, uma vez que remover casas longe do adversário raramente contribui para o seu bloqueio. Caso não existam casas adjacentes ao adversário disponíveis, o predicado recorre a qualquer casa vazia como fallback, garantindo que o algoritmo nunca falha por ausência de sucessores.

5.2.2.2 Ordenação de sucessores por score (move ordering)

O predicado moves/2 aplica uma avaliação estática rápida (staticval_fast/6) a cada sucessor gerado e ordena a lista resultante com keysort/2 antes de a devolver ao algoritmo. Para nós onde joga o maximizador ('o') a lista é devolvida por ordem decrescente de score; para nós onde joga o minimizador ('x') por ordem crescente. Esta ordenação é fundamental para a eficácia do Alfa-Beta, já que a frequência da poda depende diretamente da qualidade da ordem pela qual os candidatos são avaliados: quanto mais cedo forem encontrados valores próximos do ótimo, mais cedo os ramos inferiores podem ser descartados.

5.2.2.3 Avaliação estática otimizada

Para evitar o custo de uma nova chamada a indexOf/4 em cada avaliação, foi desenvolvida a variante staticval_fast/6, que recebe as coordenadas dos dois peões já pré-calculadas em moves/2. Complementarmente, o predicado mobilidade_at/5 substitui o padrão findall/3 seguido de length/2 por aggregate_all(count, ...), que conta as soluções diretamente sem construir a lista intermédia. Este par de predicados é chamado uma vez por sucessor durante a ordenação, pelo que a redução do custo individual tem impacto direto no desempenho global.

5.2.2.4 Corte antecipado em estados decisivos

O predicado goodenough/9 foi estendido com dois clauses adicionais que terminam imediatamente a exploração dos irmãos quando o valor do candidato atual atinge ±1000, correspondentes a estados terminais de vitória ou derrota. Quando o computador identifica uma vitória garantida ou uma derrota inevitável, não faz sentido continuar a avaliar alternativas, já que nenhum valor heurístico intermédio pode ultrapassar a magnitude de um estado terminal. Este corte opera em complemento à poda alfa-beta tradicional e é particularmente relevante nas fases finais do jogo, onde vários ramos convergem para estados terminais.

5.3 Comparação Minimax vs Alfa-Beta

Tabela 2 Comparação Minimax vs Alfa-Beta

Critério	Minimax	Alfa-Beta
Completude	Sim	Sim
Limite de profundidade	Não	Sim
Função de avaliação	Apenas terminais	Terminal + mobilidade
Velocidade em 3x3	Aceitável	Rápido
Velocidade em 6x6	Muito lento / impraticável	Aceitável

6 Avaliação Experimental

6.1 Tabuleiro 3x3 Minimax sem limite

Foram realizados testes com o tabuleiro 3x3 para validar a correção dos algoritmos, dado que o espaço de estados é reduzido. O Minimax explorou a árvore completa e o computador ('o') jogou sempre de forma correta.

- Número de estados explorados: elevado, mas gerível num tabuleiro 3x3.
- Resultado: o computador contra um humano nunca teve um resultado negativo(derrota).
- Validação: os resultados foram consistentes com o comportamento esperado do Minimax.

6.2 Tabuleiro 6x6 Alfa-Beta com profundidade 3

Para o tabuleiro padrão de 6x6, o Minimax completo tornou-se impraticável devido ao elevado fator de ramificação. O Alfa-Beta com profundidade limitada a 3 apresentou tempos de resposta aceitáveis, contidos entre 1 e 5 segundos por jogada, dependendo da fase do jogo.

- Qualidade das jogadas: o computador demonstrou comportamento estratégico, privilegiando posições com maior mobilidade própria e menor mobilidade do adversário.
- Limitação observada: com profundidade 3, o computador não antecipa situações a mais de 3 níveis de profundidade, o que pode levar a erros estratégicos em certas ocasiões.

6.3 Impacto da profundidade no Alfa-Beta

Foi testado o impacto da profundidade máxima no desempenho do algoritmo:

Tabela 3 Tabela de profundidade/tempo tabuleiro 4x4 (melhor caso)

Profundidade (Nível)	Tempo (segundos)
2	0.05
3	1
4	8
5	56
6	164

Tabela 4 Tabela de profundidade/tempo tabuleiro 4x4 (pior caso)

Profundidade (Nível)	Tempo (segundos)
2	0.4
3	5
4	48
5	Indefinido
6	Indefinido

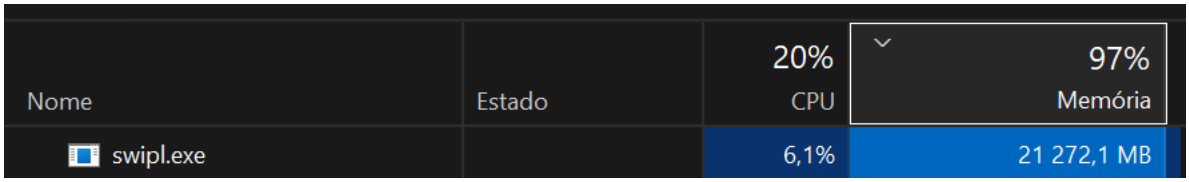

Figura 1 memória usada pelo prolog

Concluiu-se que a profundidade 3 representa um bom compromisso entre qualidade e velocidade para o tabuleiro, ressalta-se que o desempenho do algoritmo no pior caso para as profundidades 5 e 6 permaneceu indefinido, visto que necessitava de mais memória, mesmo após a expansão do limite de utilização para 20 GB.

6.4 Testes

```
Escolha onde colocar um "#" (Row/Col): |: 2/2.
Obstaculo colocado!
  0 1 2
0 . . x
1 # . #
2 . o #

  0 1 2
0 . . x
1 # . #
2 . o #

Round 3: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2
0 . # x
1 # o #
2 . . #

Player x perdeu :(
true .
```

Figura 2 Vitória do minimax

```
2 # # # #
3 # . o #

  0 1 2 3
0 # . . x
1 # # # #
2 # # # #
3 # . o #

Round 11: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2 3
0 # . # x
1 # # # #
2 # # # #
3 # o . #

  0 1 2 3
0 # . # x
1 # # # #
2 # # # #
3 # o . #

Player x perdeu :(
true .
```

Figura 3 vitória do alfa-beta

```
  0 1 2
0 # . .
1 x # #
2 o . #

Round 4: Player x, enter move (Row/Col): |: 1/1.
Jogada invalida! Tente novamente.

  0 1 2
0 # . .
1 x # #
2 o . #

Round 4: Player x, enter move (Row/Col): |:
```

Figura 4 Jogada invalida do humano

```
1 ?- play(0,[[x,.,.],[.,.,.],[.,.,o]]).

  0 1 2
0 x . .
1 . . .
2 . . o

Round 0: Player x, enter move (Row/Col): 1/1.

  0 1 2
0 . . .
1 . x .
2 . . o

Escolha onde colocar um "#" (Row/Col): |: 1/2.
Obstaculo colocado!
  0 1 2
0 . . .
1 . x #
2 . . o

  0 1 2
0 . . .
1 . x #
2 . . o

Round 1: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2
0 . . .
1 . x #
2 . o #

  0 1 2
0 . . .
1 . x #
2 . o #
```

Figura 5 Jogo perfeito por parte da AI parte 1

```
Round 2: Player x, enter move (Row/Col): |: 1/0.

  0 1 2
0 . . .
1 x . #
2 . o #

|: 1/1.
Obstaculo colocado!
  0 1 2
0 . . .
1 x # #
2 . o #

  0 1 2
0 . . .
1 x # #
2 . o #
```

Figura 6 Jogo perfeito por parte da AI part 2

```
Round 3: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2
0 # . .
1 x # #
2 o . #

  0 1 2
0 # . .
1 x # #
2 o . #

|: 1/1.
Jogada invalida! Tente novamente.

  0 1 2
0 # . .
1 x # #
2 o . #

Round 4: Player x, enter move (Row/Col): |: 1/1.
Jogada invalida! Tente novamente.

  0 1 2
0 # . .
1 x # #
2 o . #

|: 0/1.

  0 1 2
0 # x .
1 . # #
2 o . #

Escolha onde colocar um "#" (Row/Col): |: 1/0.
Obstaculo colocado!
  0 1 2
0 # x .
1 # # #
2 o . #

  0 1 2
0 # x .
1 # # #
2 o . #
```

Figura 7 Jogo perfeito por parte da AI part 3

```
Round 5: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2
0 # x #
1 # # #
2 . o #

  0 1 2
0 # x #
1 # # #
2 . o #

Player x perdeu :(
true .
```

Figura 8 Jogo perfeito por parte da AI part 4

```
scolha onde colocar um "#" (Row/Col): |: 2/2.
bstaculo colocado!
 0 1 2 3
 . . . .
 . x . .
 . . # .
 . . . o

 0 1 2 3
 . . . .
 . x . .
 . . # .
 . . . o

ound 1: O Computador (o) esta a pensar (Alpha-Beta)...
elhor avaliacao: -4
 0 1 2 3
 . . . .
 . x . .
 # . # .
 . . o .

 0 1 2 3
 . . . .
 . x . .
 # . # .
 . . o .

: nd 2: Player x, enter move (Row/Col): |:
```

Figura 9 alfa-beta com 6 de profundidade

```
Escolha onde colocar um "#" (Row/Col): |: 2/2.
Obstaculo colocado!
  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

Round 1: O Computador (o) esta a pensar (Alpha-Beta)...
  0 1 2 3
0 . . . .
1 # x . .
2 . . # o
3 . . . .

  0 1 2 3
0 . . . .
1 # x . .
2 . . # o
3 . . . .

|: nd 2: Player x, enter move (Row/Col): |:
```

Figura 10 alfa-beta com 5 de profundidade

```
Escolha onde colocar um "#" (Row/Col): |: 2/2.
Obstaculo colocado!
  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

Round 1: O Computador (o) esta a pensar (Alpha-Beta)...
Melhor avaliacao: -4
  0 1 2 3
0 . . . .
1 # x . .
2 . . # o
3 . . . .

  0 1 2 3
0 . . . .
1 # x . .
2 . . # o
3 . . . .

Round 2: Player x, enter move (Row/Col): |:
```

Figura 11 alfa-beta com 4 de profundidade

```
Escolha onde colocar um "#" (Row/Col): |: 2/2.
Obstaculo colocado!
  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

  0 1 2 3
0 . . . .
1 . x . .
2 . . # .
3 . . . o

Round 1: O Computador (o) esta a pensar (Alpha-Beta)...
Melhor avaliacao: -2
  0 1 2 3
0 . . . .
1 . x # .
2 . . # o
3 . . . .

  0 1 2 3
0 . . . .
1 . x # .
2 . . # o
3 . . . .

Round 2: Player x, enter move (Row/Col): |:
```

Figura 12 alfa-beta com 4 de profundidade

7 Decisões de Design e Justificações

7.1 Separação modular do código

A organização em ficheiros distintos (main.pl, alpha_beta.pl, utils.pl, replace.pl, print.pl) segue o princípio da separação de responsabilidades, facilitando a manutenção, os testes isolados por módulo e a substituição de componentes, nomeadamente a comutação entre o algoritmo Minimax e o Alpha-Beta, realizada através da simples alteração da diretiva consult/1 no ficheiro principal.

7.2 Mobilidade como heurística de avaliação

A função de avaliação baseada na diferença de mobilidade entre jogadores é estritamente mais informativa do que uma avaliação binária (vitória/derrota), por permitir comparar estados não terminais. Esta propriedade é essencial para o funcionamento do Alfa-Beta com profundidade limitada, uma vez que a maioria dos estados avaliados não correspondem a posições terminais. A heurística orienta consistentemente o computador para estados em que preserva opções próprias e simultaneamente restringe as do adversário.

7.3 Profundidade de pesquisa a 3

O valor foi selecionado empiricamente como ponto de equilíbrio entre tempo de resposta e qualidade das jogadas num tabuleiro 6×6. Dado o elevado fator de ramificação do ISOLA resultado da combinação entre movimento do peão (até 8 direções) e remoção de uma casa, profundidades superiores tornam o tempo de decisão impraticável sem representações de estado mais eficientes. Em tabuleiros de menor dimensão, o parâmetro pode ser aumentado sem impacto significativo no desempenho.

7.4 Validação tolerante das jogadas do utilizador

A introdução de uma jogada inválida não penaliza o jogador com a perda do turno. O predicado play/2 recorre recursivamente a si próprio com o mesmo estado, solicitando nova entrada até que seja válida. Esta abordagem melhora substancialmente a experiência de utilização e é também aplicada à colocação de obstáculos através do predicado setupSharp/2.

7.5 Utilização de findall/3 para geração de sucessores

A construção da lista completa de sucessores é delegada ao findall/3, tirando partido do modelo de execução por backtracking do Prolog de forma puramente declarativa, o predicado auxMoves/2 descreve o que é um movimento válido, e o findall/3 materializa o conjunto. Esta separação entre especificação e enumeração é idiomática em Prolog e torna o código mais conciso e verificável do que uma implementação iterativa equivalente noutras linguagens.

8 Conclusão

A realização deste trabalho prático permitiu concluir com sucesso a implementação do jogo ISOLA em Prolog, suportando tanto partidas entre humanos como contra o computador.

Do ponto de vista pedagógico, os objetivos propostos foram plenamente alcançados, proporcionando a consolidação de conceitos fundamentais da linguagem, tais como a manipulação de listas, o mecanismo de backtracking e a utilização de predicados embebidos.

No que concerne à Inteligência Artificial, o desenvolvimento do projeto demonstrou na prática a superioridade do algoritmo Alfa-Beta face ao Minimax clássico na gestão de árvores de jogo com um elevado fator de ramificação. Conclui-se que a sua aplicação, aliada a um limite de profundidade de pesquisa e a uma função de avaliação heurística focada na mobilidade, foi determinante para garantir um agente inteligente capaz de tomar decisões estratégicas com tempos de resposta adequados.